4 Implementation

4.1 Description

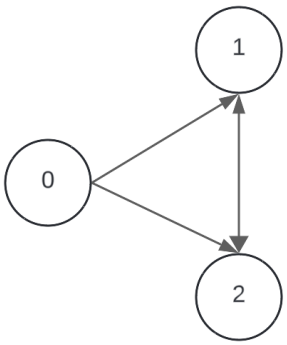
We made a program that models the puzzle as a dynamic epistemic model. The states represent a collection of configurations that remain possible based on what has been publicly announced and publicly observed. The visibility matrix defines who can see who and is assumed to be known to every agent and known to be known (agents can reason both from their observations and from what they know other agents can or cannot see), which is important once you have imperfect visibility, since an agent can obtain knowledge from another's lack of action, even if the second agent had insufficient visual information to take an action themselves.

Our model has a new public announcement every single day. The first announcement on Day 0 is "there is at least one blue-eyed agent". This eliminates the possibility of there being no blue-eyed agents. From Day 1 forward, the public knowledge is the fact that everyone leaves at the same time. We model the incorporation of this public knowledge as a public announcement that removes the possibilities that could have led to a different behavior than was observed. In particular, our implementation models the removal of a possibility as removing it for everyone at the same time, which allows for the use of higher order reasoning regarding others' reasoning to model the ability of each agent to reason iteratively every step.

4.2 Worlds and Visibility

A world is an assignment of eye colours to all of the agents in the system. Internally, we represent these worlds compactly using a binary code where every agent is associated with a single bit in the binary code that represents that agent's eye colour (blue or brown). This is equivalent to our formal model of how the world is described (since it directly describes each agent's eye colour). This form of representation has advantages in practice, particularly when we consider the fact that the primary function of the simulator is to continually restrict the set of possible worlds based on the current state of the system. Using a compact form of representation allows us to do this quickly and easily while still allowing for inspection of the restricted worlds.

We use a binary matrix E to represent the visibility structure of the agents in the system. When we say $E[i][j] = 1$, we mean that agent j can observe agent i's eye colour as seen in Figure 8. The diagonal entries are usually set to 0 to indicate that agents cannot observe their own eye colours. However, to allow for exploration of edge cases variants, we keep the representation



	Agent 0	Agent 1	Agent 2
Agent 0	☐	☐	☐
Agent 1	☑	☐	☑
Agent 2	☑	☑	☐

Figure 8: Connected graph of what eyes agent j (column) can see given a visibility matrix.

general. E is the source of information from which the agents' ability to observe other agents' eye colours is determined. Its structure will determine the entire update process.

4.3 Knowledge

We do not compute all the accessibility edges of a Kripke model. The implementation determines what j knows using the indistinguishable worlds that it produced from the visible information. We define two worlds as being indistinguishable from agent j's point of view iff they have the same colour of eyes for all agents j can see, because then j will observe the same amount of visual information in those worlds. The implementation uses an "observation signature" to represent this computation. Each world represents what j can see, represented as a string of bits based upon the eye colours of all other agents j can see. Worlds having the same observation signature represent worlds that j views identically and thus represent the worlds that j views as possible worlds. An agent can deduce knowledge from its own candidate worlds and does not simply use all possible worlds. An agent j will know they are blue in a candidate world w if j is blue in every candidate world that shares j's observation signature. It is not sufficient that "j is blue in all currently possible worlds", because different agents observe differently. What matters is whether the possibilities which remain consistent with what the agent sees allow for the possibility that j is non-blue. This step makes sure we have a practical test that the simulator can perform after each new day. At any given time, there are three options for each agent: one where there is certainty that the agent is blue, one where there is certainty that the agent is not blue or one where there is no conclusion either way (see Figure 9).

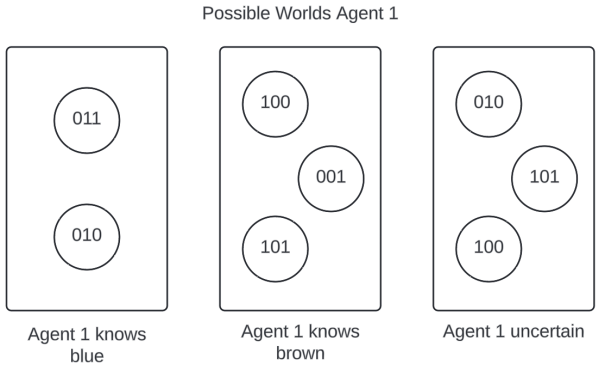


Figure 9: Two different possibilities for agent 1 to deduce knowledge and one to remain uncertain given a possible world set.

4.4 The Day-By-Day Engine

Each day of the simulation uses a public update that all of the agents use at the same time. Day 0 is used to implement the outsider's announcement that there is at least one blue-eyed agent by narrowing down the candidate set to the worlds in which at least one of the blue propositions of each agent are true. After day 0, no additional spoken statements are necessary for creating new information. The new information created after day 0 occurs with the publicly observable pattern of who left and who did not. That pattern is treated as common knowledge since it is a public event that all agents use for their updates and they know that every other agent will also update in the same manner.

The operation of how the update works is to first determine, in each candidate world, which agents would have left that world if that world had been the actual world. Agents will leave when they know they are blue. The actual world used to select the run determines which leaving pattern is observed that day. The next step in the process is the restriction step where only the candidate worlds that match the observed leaving pattern are retained. If a candidate world would cause an agent to know that they are blue and therefore leave but that agent does not actually leave, then that candidate world is no longer possible and is eliminated. Imperfect visibility makes this effect more subtle. No one leaving may be informative but why it is informative will depend upon what everyone knows about what everyone else can see.

A run can be classified as solved when in the real world each blue-eyed person will know they are blue and thus leave. In the implementation we look at the observed leaving set and if it contains all blue-eyed persons. If it does, we classify this as a solved run. If no more worlds may be eliminated through further public updates and there are still blue-eyed agents who remain uncertain about their colour, then we classify the run as stuck. The model will reach a fixed point and additional days cannot provide additional eliminations from the candidate world set. In practice, our implementation adds a max-days safety bound to limit the number of days of the run. However, the concept of a stuck run is the stabilization of the candidate world set, not the number of days until it reaches some arbitrary limit. It is here that the behavior of the solver becomes apparent. The solver does not guess or search for a solution by testing various hypotheses. It iteratively applies the public update rule and determines whether the structure of indistinguishable worlds will allow for further worlds to be eliminated.

4.5 Analysis

There are two different ways to utilize this implementation. Single instance mode is used to simulate the PAL day cycle using one visibility matrix and one selected actual world. The mode is used to show how bad visibility may delay or prevent the emergence of certainty (how long it will take for an agent to become certain). In the matrix analysis mode we have two different modes to study the solvability of the matrices. Instance solvability examines if a given matrix has an instance solution for one selected actual configuration of agents. Uniform solvability determines if a matrix is guaranteed to have solutions for all possible non-empty configurations of blue-eyed agents. Once a matrix is determined to be uniformly solvable, it can be classified as solvable. The uniform solver represents the interpretation of the original riddle as it is testing if convergence is a result of both the inherent properties of the visibility structure and the mechanisms of public reason as opposed to just being a result of lucky positioning of the agents blue eyes.

To verify the technical correctness of this solution it was tested with typical patterns from the benchmark behaviour of the original problem to confirm that it works under known conditions. The fully visible version of the simulation produced the expected result as the original problem, where if there are k blue-eyed agents, then no agents will exit for $k-1$ days and then all blue-eyed agents will exit on day k. We also ran standard test cases such as the single agent case (where the first day gives the final result) and non-visible cases (where there is no information that can be exchanged between agents).